Astra and Grok: an orchestration plan

One planner, visible workers, durable communication

Prepared for Paul | 13 September 2026 | Proposed design

Build a small local runtime controlled by Astra. Astra owns planning, delegation, routing decisions, and acceptance. Grok workers execute bounded tasks and report only to Astra. Runtime code handles persistence, delivery, and process monitoring without consuming a model turn for routine bookkeeping.

Keep two logical structures: **current state** and **append-only history**. Inboxes and the bulletin are views over them. Central supervision is an established pattern [1]; these particular schemas and guarantees are proposals. Live transport, interruption, and recovery must pass the implementation gates on page 6.

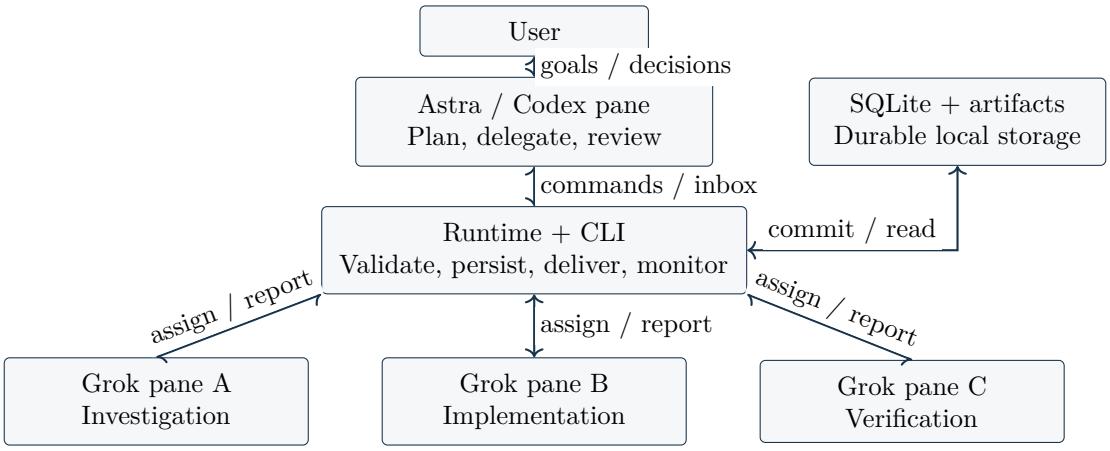


Figure 1. Service relationships. The runtime transports messages; Astra makes coordination decisions. Worker-to-worker messaging is absent by design.

Component	Responsibility
Orchestrator skill	Delegation and review policy; scoped assignments, relevant context, dependency decisions, and result acceptance.
Runtime service	One local daemon with storage, delivery scheduler, process manager, and provider adapters as modules. No separate broker in v1.
Worker runner	Owns the Grok process in its pane; reports events, receives assignments, and preserves logs. No nested agents in v1.
Artifact store	Versioned files for contracts, code evidence, and large outputs. Messages carry references with content hashes.

Data structures and ownership

Stable IDs outlive processes. Keep `worker_id`, `task_id`, and `run_id` distinct. A task may be reassigned; a worker may restart; a pane may be reused. PID, process start identity, pane ID, and Grok session ID are handles, not primary keys. A retry creates a new run and retains the old history.

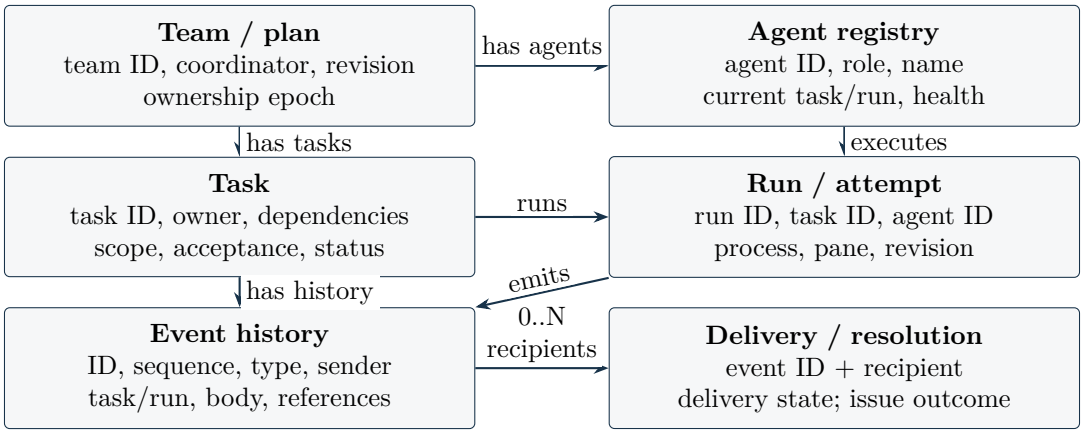


Figure 2. Main record relationships. Event history is immutable; the other records hold current state. All records are scoped to one team.

Requested view	Proposed representation
Process mapping	`orchestrator` and `workers`, keyed by stable agent ID. Show name, current task/run, health, last heartbeat, and last substantive event.
Alive / dead	Derived filters over one registry. Do not move records between separate dictionaries; unresponsive is distinct from exited.
Work history	One ordered event stream, filtered by orchestrator, worker, task, or run. Preserve timestamps and sequence numbers as fields.
Normal / override inbox	Pending or unresolved deliveries for Astra, filtered by priority. Each worker also has an inbox for Astra-authorized commands.
Community bulletin	Published announcements with source proposal and recipient references. Actionable announcements also create tracked deliveries.

Write contract. The runtime atomically appends the event, changes affected state, and inserts pending deliveries before returning a durable receipt. SQLite provides transactional isolation with serialized writers [3]. Agents use the CLI/API; they do not rewrite a shared JSON document. JSON remains the inspection/export format. Artifact files are finalized and hashed before their references are committed; incomplete or missing artifacts cannot satisfy acceptance.

Message schema and delivery contract

Structured metadata; free-form explanation. The example below is a persisted event, not a delimiter-based string. The runtime assigns event IDs, team sequence numbers, timestamps, and validated sender identity. Workers supply the report and suggested recipients; only Astra can authorize forwarding.

```
{
  "schema_version": 1,
  "event_id": "evt-0042",
  "team_id": "auth-upgrade",
  "sequence": 42,
  "timestamp": "2026-09-13T14:05:00Z",
  "type": "plan_conflict",
  "from": "worker-auth",
  "run_id": "run-017",
  "to": "orchestrator",
  "task_id": "task-auth-api",
  "plan_revision": 3,
  "priority": "urgent",
  "suggested_recipients": ["worker-ui", "worker-tests"],
  "body": "The contract cannot represent partial success.",
  "artifact_refs": ["artifact-contract-v2"],
  "reply_to": null
}
```

Message	Direction	Meaning
Assignment	Astra to worker	Task, scope, dependencies, revision, acceptance criteria.
Progress / result	Worker to Astra	Store progress quietly; a result enters review.
Question / conflict	Worker to Astra	Track unresolved issue; priority can be urgent.
Proposal	Worker to Astra	Suggest recipients and attach an artifact reference.
Publication	Astra to workers	One delivery per selected recipient; retain source event.
Pause / resume / cancel	Astra to runner	Track requests separately from confirmed outcomes.

Four distinct facts. A *receipt* confirms durable storage. *Delivered* means the adapter submitted the message to the session. *Acknowledged* means the recipient accepted that message ID, confirmed by the protocol or explicit worker response. *Resolved* means the question or requested action has an outcome. Acknowledgment never proves task completion. Established messaging systems make the same receipt/processing distinction [4].

Each mutation carries a request ID. An identical retry returns the same receipt; changed content with the same ID is rejected. Delivery retries reuse the message ID and recipients deduplicate. Track delivery per recipient, not only a high-water timestamp: urgent processing may skip older normal messages. Sequence numbers record commit order, not a guarantee of causal order between independent workers.

Normal assignments and mediated broadcasts

Workers run while Astra remains available. The runtime presents compact results and actionable issues. Routine progress, heartbeats, and delivery bookkeeping are recorded without triggering conversational acknowledgment loops.

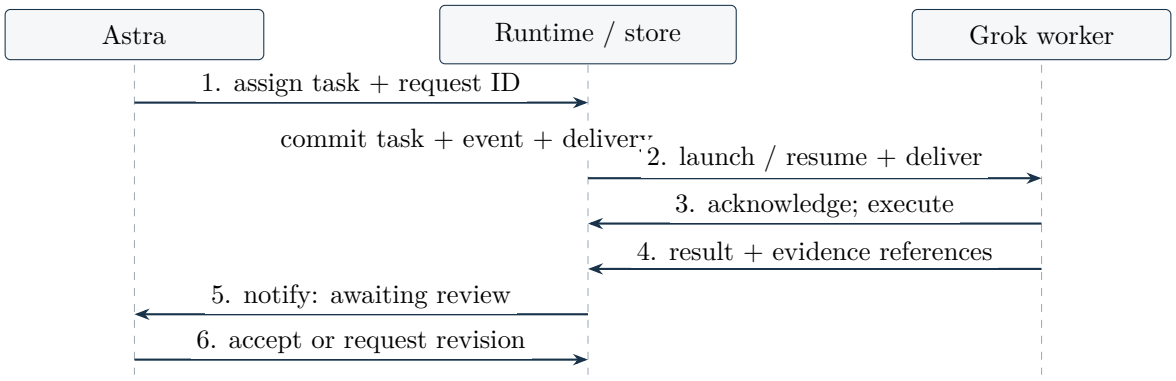


Figure 3. Assignment and result flow. The runtime records Astra's acceptance decision. Process exit or a zero exit code alone cannot accept a task.

The bulletin records approved routing. A worker proposes an announcement; Astra determines whether it affects the plan and chooses recipients. Forward a versioned contract reference rather than paraphrasing the contract. A bulletin entry alone is insufficient when another worker must act: create a delivery too.

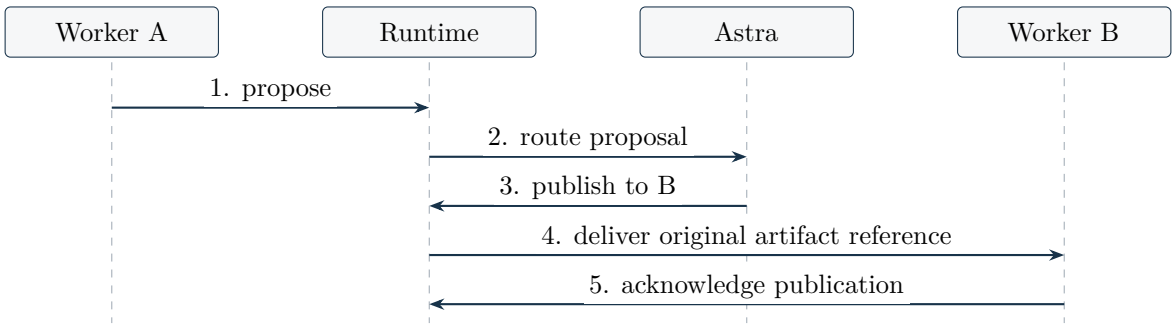


Figure 4. One recipient is shown. Fan-out produces separate delivery records while retaining one publication and its original proposal.

Context contract. Each assignment includes the objective, relevant plan decisions, expected output, acceptance checks, and exclusive write scope. Workers return findings, artifact references, validation evidence, and unresolved issues. Astra retrieves history selectively after compaction instead of rereading all logs.

Urgent messages and recovery

Priority changes scheduling, not storage order. The override inbox is the urgent unresolved view. It is checked before normal messages; repeated reports of the same issue are coalesced. An urgent flag alone cannot interrupt a model turn or stop an executing tool. The transport adapter must establish those capabilities.

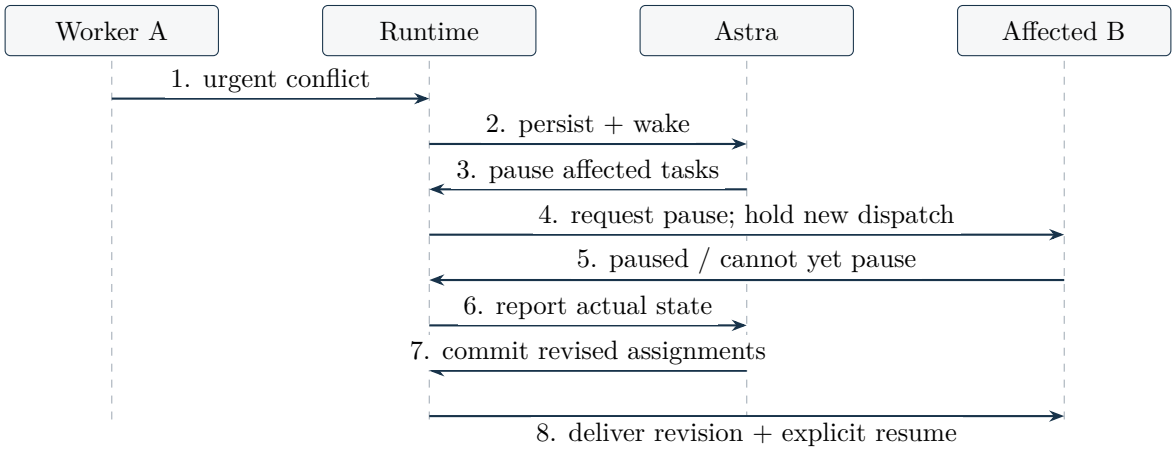


Figure 5. Urgent replan. Worker A stops its own dependent work when blocked. Astra selects affected tasks; unrelated work can continue. A pause is never assumed before confirmation.

Failure or race	Required behavior
Old-plan result arrives	Preserve it with its assignment revision. Astra checks changed dependencies before accepting or requesting revision.
Worker exits or stalls	Separate process health from task outcome. Inspect artifacts and pending actions before retrying with a new run ID.
Runtime / Astra restarts	Restore plan, unresolved deliveries, and runs. Reconcile actual process identities and sessions before new dispatch.
Delivery ack is lost	Retry the same message ID; recipient deduplicates. If acceptance cannot be verified, preserve the outcome as uncertain.
External action times out	Inspect the original action before retrying. Message deduplication does not guarantee exactly-once external effects.

Recovery ownership. One active coordinator session controls a team. An ownership epoch rejects stale-session commands after takeover. Record execution intent before launching a process and observed outcome afterward; reconcile incomplete intents on restart. Durable history remains outside the agent's context, consistent with the recovery principle in [2].

Independent states. Tasks move from queued to running to awaiting review to accepted, with blocked, failed, and cancelled outcomes. Run health is starting, running, unresponsive, or exited. Pause/cancel requests are control states with separate confirmation. A live PID does not imply useful progress or completed work.

Delivery plan, tmux rules, and decision gates

First milestone: two tasks and one revision in a single window. Astra delegates two tasks, receives results, requests one revision, and accepts the combined outcome. Also demonstrate restart recovery and an urgent replan.

	Atomic stage	Exit evidence
1	Transport spike	Launch, stream, acknowledge, follow up, resume, and cancel one Grok run. Wake the exact Astra session; document unsupported behavior.
2	Store + pane lifecycle	Stable IDs, transactional events and deliveries, pane ownership, reuse, and tiling. No lost concurrent events or wrong-pane cleanup.
3	Routing + recovery	Normal/urgent inboxes, bulletin fan-out, deduplication, takeover, and stale-result checks. Recover at each delivery boundary.
4	Skill + integration	Astra/Grok role contracts, isolated worktrees, acceptance review, and a CI-passing Graphite stack. Complete the first milestone.

Pane contract. Proposed default: the orchestrator's existing tmux window, with a separate pane per agent and a visible runtime/status pane. Register exact server, window, and pane IDs. Reuse idle team-owned panes; otherwise split and run `select-layout tiled`, the action already bound to Ctrl-b then `t`. Preserve focus; cleanup only the worker pane. Queue work when the screen is full. Verify process start identity before signaling a PID. Reuse sessions for related work; start fresh sessions for unrelated work.

Code ownership. One current executing attempt per task; isolated worktrees for concurrent writers. One named integration worker assembles accepted changes. Use `gt create` and `gt submit -no-interactive`; every PR must independently pass required CI. Publication stays within the task's authorization.

Open choices and local evidence. Recommend a live runner activity view with steering commands; full Grok TUI remains an open choice. Start with two workers. Installed Grok 1.0.30 advertises streaming JSON; Codex 0.154.0 advertises `queue`. Interruption semantics remain untested. Existing `orch` creates and kills separate windows; reuse selectively. Omit its mandatory `poteto-mode` and nested agents. Create isolated checkouts with `git worktree add` before code tasks.

Required fault tests. PID reuse; crash before send and after send/before ack; duplicate assignment; urgent conflict during a tool call; old-revision result; coordinator takeover. Measure urgent acknowledgment latency, recovery time, failed deliveries, and coordination turns before increasing concurrency.

Evidence and limits

1. LangChain, Subagents, "Key characteristics" and "Sync vs. async." Supports centralized routing and asynchronous workers, not this exact implementation.
2. Anthropic, Scaling Managed Agents: Decoupling the brain from the hands, 8 April 2026, "Decouple the brain from the hands" and "The session is not Claude's context window." Supports durable history independent of a running harness.
3. SQLite, Isolation in SQLite, "Isolation and Concurrency." Transactional isolation and serialized writers.
4. RabbitMQ, Consumer Acknowledgements and Publisher Confirms. Durable receipt and recipient processing are separate. No RabbitMQ dependency is proposed.

Sources retrieved 13 September 2026. Local source copies, CLI help, hashes, and line references accompany this plan in `evidence/`. All proposed runtime guarantees remain subject to the above tests; no workers were launched for this plan.